DESKTOP UX GUIDE

HYPERCLUSTER DESKTOP ? UX GUIDE

Liquid Glass desktop UI for the hypercluster CLI. Every cluster action runs through the hypercluster binary ? the app never calls helm, kubectl, or SSH directly for lifecycle operations.
Launch: npm run desktop from repo root, or hypercluster-desktop after global install.

Table of Contents

1. Getting Started
2. Navigation
3. Dashboard & cluster detail
4. Cluster Creation Studio
5. Advanced operations
6. Deploy & Log Console
7. Settings
8. Command Palette & global search
9. Keyboard Shortcuts
10. Notifications & Tray
11. Data Storage
12. CLI Mapping Reference

1. Getting Started

Install and run:

```
cd hypercluster
npm install
npm run desktop
```

Global install (optional):

```
npm install -g ./desktop
hypercluster-desktop    # alias: hc-desktop
```

First launch checklist:

1. Open **Settings** and confirm the resolved **hypercluster binary** path (repo ./hypercluster is auto-detected in dev).
2. **Import** an existing cluster.conf or open **Create Cluster Hub** (Quick Deploy or Studio).
3. Run **preflight** before first **deploy**.

Auto-import:

On startup, if the configured hypercluster binary lives in a repo that contains cluster.conf, the app imports that cluster once (when the cluster name is not already in the registry).

2. Navigation

The left sidebar has four views:

View · Purpose
Dashboard · Cluster cards, stats, import, quick deploy
Cluster detail · Sidebar entry for the selected cluster (nodes, tests, preflight)
Create Cluster · Intent hub ? Quick Deploy or Cluster Studio (ClusterSpec ? cluster.conf)
Advanced ops · Destroy, remove-node, dry-run deploy
Settings · Binary path, registry list with config paths, import
The footer shows **?K command palette** hint (Ctrl+K on Windows/Linux).

3. Dashboard & cluster detail

Dashboard:

- Shows resolved **hypercluster binary** path.
- **Import cluster.conf** ? native file picker; copies config into app registry.
- **+ New Cluster** ? opens the **Create Cluster Hub** (intent cards ? Quick Deploy or Studio).

Stat cards:

Card · Meaning
Clusters · Count in local registry
Ready nodes · Sum of ready nodes (status --json per cluster)
Active job · 1 while a hypercluster subprocess is running

Cluster cards:

Each card shows:

- Cluster **name** and **distro** (k3s or kubespray)
- **Status pill** ? unknown / ready / running (active job) / error (last command failed)
- **Config path** ? truncated registry path
- **Reveal cluster.conf** ? opens config in Finder (macOS), Explorer (Windows), or file manager (Linux)
- **Nodes ready** ? e.g. 1/1 nodes ready when kubeconfig exists
- **Open** ? **Cluster detail** view (nodes, smoke tests, preflight panel)
- **Deploy** ? runs hypercluster deploy with live logs

3.1 Cluster detail (cockpit)

Opened from a cluster card **Open** button or the command palette.
The left sidebar lists **14 cockpit sections**: Overview, Nodes, Workloads, Networking, Storage, Platform, VMs, Templates, GPU, Security, Observability, Backups, Upgrades, Settings.
Section · Source · Notes
Overview · Registry + status --json · Install plan from ClusterSpec
Nodes · status --json · Ready counts, roles
Workloads · workloads --json · Pods and services (requires kubeconfig)
Platform smoke · test --json · Run test / test --full
Preflight · preflight --json · SSH and tool checks
VMs / Templates · vms --json · KubeVirt VMIs + catalog; template create is preview stub
Settings · ? · **Open Cluster Studio** (single edit path on ClusterSpec)
Toolbar: preflight, deploy, test, test --full, **Edit conf** (Monaco).
Remove from registry deletes the desktop entry only (does not run destroy). Type the cluster name to confirm.

3.2 Monaco cluster.conf editor

Edit conf (detail view or Operations) opens the Monaco editor for the registry cluster.conf.
- **Save** writes via the Tauri backend and refreshes the registry.
- **Cancel** returns to the previous view (detail or Operations).
After saving, a prompt offers **Run preflight** to validate changes.
Keyboard: **?S** / **Ctrl+S** save · **Esc** cancel.

Empty state:
When no clusters are registered, a glass panel prompts you to import cluster.conf or open the Create Cluster Hub.

4. Cluster Creation Studio

The typed **ClusterSpec** model (desktop/src/lib/clusterSpec.ts) is the single edit path. Generated cluster.conf is written on save/deploy.
Flow · Steps · Use when
Quick Deploy · Name ? Infrastructure ? Deploy · Fast path with template defaults
Cluster Studio · Intent ? Nodes ? Network ? Platform ? Validate · Basic / Advanced / Expert tiers
Import · File picker · Existing external config
Save: new clusters register under app data; **editing** updates the same config_path (registry UUID preserved).
Settings search (Studio): wired fields only ? catalog items (Kyverno, Velero, ?) are read-only.
Per-node SSH overrides persist as NODE_SSH_OVERRIDES_JSON in cluster.conf (k3s deploy path).

Related CLI (JSON for Desktop):
Command · Purpose
discover --json --source ? · Node discovery (libvirt, Proxmox, CSV, ?)
vms --json · KubeVirt VMIs + templates
vm create --template ? --name ? --json · Template validation (create stub)
workloads --json · Pod/service inventory in cockpit

5. Advanced operations

Infrequent or destructive commands. Common tasks (preflight, deploy, test, edit conf) are on **cluster detail**.
Button · hypercluster command
dry-run deploy · deploy -n
remove-node · remove-node <name> -f
Destroy cluster · destroy -f (type cluster name to confirm)
Use **Open cluster detail** for day-to-day work.

6. Deploy & Log Console

When any command runs, the view switches to **Deploy** (log console):
- **Title** ? cluster name + command (+ dry-run suffix when applicable)
- **Stage card** ? parsed deploy stage (current/total + label) when hypercluster emits stage lines
- **Log console** ? streaming stdout/stderr (ANSI stripped for display)
- **Cancel job** ? sends SIGTERM to the running subprocess
- **Back to cluster detail** ? after a job started from detail finishes; panels refresh automatically (Esc also works)
- **Success/failed** badge in the log view title when the job completes

On completion, a native **notification** shows success or failure.
The dashboard **Active job** stat reflects running state.

7. Settings

Control · Action
Custom hypercluster binary path · Override auto-detected binary
Save · Persists to registry settings
Import cluster.conf? · Same as dashboard import (lands on cluster detail)
Open data directory · Opens app data folder in the system file manager
Registered clusters · List with distro, config path, status pill, **Reveal conf**

8. Command Palette & global search

Command palette (?K / Ctrl+K):
- Search filters clusters by name, distro, or config path; filters commands and navigation actions.
- Per cluster: **Open cluster detail**, **Edit cluster.conf**, **Open Cluster Studio**, plus run commands (preflight, deploy, test, test --full, status, fetch-kubeconfig, merge-kubeconfig, unlock).
- **Escape** or **?K** again closes the palette.
Selecting a command starts the job and opens the log console.

Global search (??F / Ctrl+Shift+F):
Cross-view search for clusters, navigation targets, and studio actions. Complements the command palette for discovery when you know a feature name but not which cluster owns it.

9. Keyboard Shortcuts

Shortcut · Action
?K / **Ctrl+K** · Toggle command palette
??F / **Ctrl+Shift+F** · Global search
?S / **Ctrl+S** · Save in Monaco editor
Escape · Close palette / settings search; from log view return to cluster detail when job finished

10. Notifications & Tray

- **macOS menu bar tray** ? Open Hypercluster / Quit
- **Job notifications** ? title "Hypercluster", body includes command name and result
- **Tray click** ? show and focus main window

11. Data Storage

Path (macOS) · Content
~/Library/Application Support/Hypercluster/clusters.json · Registry + settings
~/Library/Application Support/Hypercluster/clusters/<name>/cluster.conf · Per-cluster config
Windows paths use the Tauri app data directory (see Settings ? Open data directory).

12. CLI Mapping Reference

The desktop invokes:

```
exec "<hypercluster>" -c "<config_path>" [-n] [-v] [-f] <command> [args...]
```

JSON used internally (not shown in log view):

Desktop surface · hypercluster command
Dashboard node counts · status --json
Cluster detail smoke · test --json / test --full --json
Cluster detail preflight · preflight --json
Workloads cockpit · workloads --json
KubeVirt VMs / templates · vms --json
Template create (preview) · vm create --template ? --name ? --json
Node discovery (Studio) · discover --json --source ?
Human-readable test output appears in the log console; JSON mode keeps stdout clean for parsers.

ClusterSpec and save behavior:

- **New cluster:** create_cluster_cmd writes clusters/<name>/cluster.conf under app data.
- **Edit existing:** update_cluster_by_path_cmd merges via mergeClusterSpec (preserves comments and registry UUID).
- **Per-node SSH:** optional NODE_SSH_OVERRIDES_JSON in cluster.conf (k3s deploy honors per-IP user/key/port).

Related documentation

- CLI guide
- Configuration reference
- Main README
- Documentation hub

Doc version: 2026-06-05T00:05:12Z · hypercluster v0.1.0